PROJEKTDOKUMENTATION

Raumbuchungssystem für Programmierkurse

Projekt	Raumbuchungssystem
Auftraggeber	Provadis GmbH
Projektteam	4 Auszubildende
Zeitraum	2 Wochen
Datum	6.3.2026
Version	1.0

Technologien: Python · Flask · SQLite · React · TypeScript

Inhaltsverzeichnis

1. Projektbeschreibung

1.1 Ausgangssituation

Im Rahmen eines zweiwöchigen Programmierkurses wurde ein Team aus vier Auszubildenden damit beauftragt, ein Raumbuchungssystem für eine Firma zu entwickeln, die Programmierkurse anbietet. Das System soll den administrativen Aufwand bei der Kursplanung erheblich reduzieren.

1.2 Aufgabenstellung

Das Raumbuchungssystem soll folgende Kern Entitäten verwalten:

- Trainer, die Kurse leiten
- Auszubildende (Azubis) – Kursteilnehmer
- Module – Lerninhalte, die in Kursen vermittelt werden
- Räume – Physische Schulungsräume mit Kapazitäten
- Kurse – Kombination aus Trainer, Modul, Raum und Zeitslot

Das System soll mehrere Benutzerrollen managen können.

1.3 Zielsetzung

Das System soll automatisch einen konfliktfreien Kursplan für ein gesamtes Schuljahr generieren. Dabei müssen folgende Konflikte vermieden werden:

- Doppelbelegung eines Raumes zur gleichen Zeit
- Überschneidung von Trainerterminen
- Überschneidung von Azubi-Kursen
- Möglichst keine Kurse in Schulferien
- Keine Kurse während Azubi Berufsschulblock

Zusätzlich soll der generierte Plan manuell angepasst und erweitert werden können.

© 2025 – Auszubildende Programmierprojekt

2. Projektplanung

2.1 Zeitplan

Das Projekt wurde in einem zweiwöchigen Sprint mit täglichen Stand-ups umgesetzt:

Woche / Tag	Phase	Aufgaben
Woche 1, Tag 1	Analyse & Setup	Anforderungsanalyse, Datenbankschema entwerfen, Projekt-Setup
Woche 1, Tag 2–3	Backend-Grundgerüst	SQLite-Datenbank aufsetzen, Fake-Daten generieren, Basismodelle
Woche 1, Tag 4–5	API-Entwicklung	REST-Endpunkte für CRUD-Operationen implementieren
Woche 2, Tag 1–2	Planungsalgorithmus	Automatische Kursplan-Generierung mit Konfliktprüfung
Woche 2, Tag 3–4	Frontend	React-Oberfläche, Kalenderansicht, manuelle Bearbeitung
Woche 2, Tag 5	Test & Dokumentation	Integrationstests, Bugfixes, Dokumentation fertigstellen

2.2 Aufgabenverteilung

Azubi	Hauptverantwortung	Aufgaben
Aleandro *“Chief Aesthetic Officer of Chromatic Fluidity”*	Frontend mit Schwerpunkt auf Design	Frontend Pages Designen, Bauen von Views und Komponenten
Sebastian *“Grand Vizier of Full-Stack Synchronicity”*	REST-API / Middleware, Frontend mit Schwerpunkt auf Anforderungserfüllung, Architekturplanung, Dokumentation	Flask Controller, JSON-Serialisierung, Planen von Frontend Aufbau, Datenformatierung
Ali *“High Chancellor of Schematic Genesis & Structural Truth”*	Dokumentation, Aufgabenstellung, Architektur	Schreiben von Dokumenten, erstellen von UML Diagrammen,
Eren *“Supreme Warden of Database Integrity & Structural Evolution”*	Backend, Datenbank, Architektur	Aufsetzen und Befüllen der Datenbank, Planung der Datenstruktur, Erstellen der Kurserstellungs algorithmen

© 2025 – Auszubildende Programmierprojekt

3. Technische Architektur

3.1 Systemübersicht

Das System folgt einer klassischen 3-Schichten-Architektur:

Frontend	Middleware (API)	Backend
React + TypeScript	Python FlaskREST-Controlle	SQLite Datenbank, Python

Die Kommunikation zwischen Frontend und Backend erfolgt ausschließlich über HTTP REST-Schnittstellen mit JSON als Datenformat. Der Flask-Server läuft lokal und stellt alle Endpunkte bereit.

3.2 Technologiestack

Schicht	Technologie	Begründung
Frontend	React 18 + TypeScript	Komponentenbasiert, typsicher, gute Kalender-Libraries
Middleware	Python Flask	Leichtgewichtig, einfache REST-API-Entwicklung
Datenbank	SQLite	Serverlos, keine Installation, ideal für Entwicklung
Backend	Python 3.x	Gute DB und Zeit Libraries

3.3 Planungsalgorithmus

Der Algorithmus zur automatischen Kursplan-Generierung läuft serverseitig in Python und arbeitet nach folgendem Prinzip:

- Alle Module, Trainer, Räume und Azubi-Gruppen werden aus der Datenbank geladen
- Für jedes Modul wird ein Trainer zugewiesen
- Es wird ein freier Zeitslot gesucht, bei dem gilt: Raum frei UND Trainer frei UND Azubis frei
- Der gefundene Slot wird eingetragen und für alle Beteiligten als belegt markiert

4. Datenbankschema

4.1 Übersicht der Tabellen

Die SQLite-Datenbank besteht aus folgenden Haupttabellen:

Tabelle	Beschreibung	Wichtige Felder
trainer	Alle Trainer der Firma	id, name, email, qualifikation
azubi	Auszubildende (Teilnehmer)	id, name, email, jahrgang
modul	Kursmodule / Lerninhalte	id, name, dauer_stunden, beschreibung
raum	Schulungsräume	id, name, kapazitaet, ausstattung
kurs	Geplante Kurstermine	id, modul_id, trainer_id, raum_id, start, ende
kurs_azubi	Zuordnung Azubi ↔ Kurs	kurs_id, azubi_id
buchung	Raumbuchungen	id, raum_id, kurs_id, datum, start_zeit, end_zeit

4.2 Beziehungen

- Ein Kurs hat genau einen Trainer, ein Modul und einen Raum (n:1)
- Ein Kurs kann mehrere Azubis haben, ein Azubi kann mehrere Kurse besuchen (n:m via kurs_azubi)
- Jede Buchung referenziert einen Raum und einen Kurs

© 2025 – Auszubildende Programmierprojekt

5. API-Dokumentation (REST)

5.1 Allgemeines

Die API läuft standardmäßig auf http://localhost:5000. Alle Anfragen und Antworten verwenden JSON. Die Basis-URL für alle Endpunkte ist /api/v1.

5.2 Endpunkte

Trainer

Methode	Endpoint	Beschreibung	Response
GET	/api/v1/trainer	Alle Trainer abrufen	200 + Liste aller Trainer
GET	/api/v1/trainer/{id}	Einzelnen Trainer abrufen	200 + Trainer-Objekt
POST	/api/v1/trainer	Neuen Trainer anlegen	201 + erstelltes Objekt
PUT	/api/v1/trainer/{id}	Trainer aktualisieren	200 + aktualisiertes Objekt
DELETE	/api/v1/trainer/{id}	Trainer löschen	204 No Content

Räume

Methode	Endpoint	Beschreibung	Response
GET	/api/v1/raeume	Alle Räume abrufen	200 + Liste aller Räume
POST	/api/v1/raeume	Neuen Raum anlegen	201 + erstelltes Objekt
PUT	/api/v1/raeume/{id}	Raum aktualisieren	200 + aktualisiertes Objekt
DELETE	/api/v1/raeume/{id}	Raum löschen	204 No Content

Kurse & Planung

Methode	Endpoint	Beschreibung	Response
GET	/api/v1/kurse	Alle Kurstermine abrufen	200 + Kursliste
POST	/api/v1/kurse	Neuen Kurs manuell erstellen	201 + Kurs-Objekt
PUT	/api/v1/kurse/{id}	Kurstermin bearbeiten	200 + aktualisierter Kurs
DELETE	/api/v1/kurse/{id}	Kurstermin löschen	204 No Content
POST	/api/v1/planung/generieren	Kursplan auto. generieren	200 + generierter Plan
GET	/api/v1/planung/konflikte	Konflikte prüfen	200 + Konfliktliste

6. Benutzerhandbuch

6.1 Systemstart

Voraussetzungen: Python 3.x, Node.js und npm müssen installiert sein.

Backend starten

- In das Backend-Verzeichnis wechseln: cd backend
- Abhängigkeiten installieren: pip install -r requirements.txt
- Server starten: python app.py
- Der Server läuft auf: http://localhost:5000

Frontend starten

- In das Frontend-Verzeichnis wechseln: cd frontend
- Abhängigkeiten installieren: npm install
- Entwicklungsserver starten: npm run dev
- Die Anwendung öffnet sich auf: http://localhost:5173

6.2 Kursplan automatisch generieren

Die automatische Plangenerierung erstellt für das gesamte Schuljahr alle Kurstermine ohne Konflikte.

1. Im Hauptmenü auf den Button „Kursplan generieren" klicken
2. Das Schuljahr auswählen (z.B. 2024/2025)
3. Auf „Generieren" klicken – das System berechnet automatisch alle Termine
4. Der generierte Plan wird in der Kalenderansicht angezeigt
5. Etwaige Konflikte oder Warnungen erscheinen in der Statusleiste

6.3 Termine manuell bearbeiten

Bereits geplante Termine können im Kalender direkt bearbeitet werden:

- Auf einen Termin klicken → Detailansicht öffnet sich
- Auf „Bearbeiten" klicken → Felder für Raum, Trainer, Zeit anpassen
- Das System prüft automatisch auf Konflikte beim Speichern
- Bei Konflikt: Fehlermeldung mit Hinweis auf die betroffene Ressource

6.4 Neuen Termin eintragen

- Auf das „+" Symbol im Kalender klicken oder „Neuer Termin" im Menü
- Datum, Uhrzeit, Raum, Trainer und Modul auswählen

© 2025 – Auszubildende Programmierprojekt

- Azubis dem Kurs zuweisen
- Auf „Speichern" klicken – Konfliktprüfung erfolgt automatisch

7. Tests

7.1 Teststrategie

Im Rahmen des zweiwöchigen Projekts wurden folgende Testarten durchgeführt:

Testart	Beschreibung	Werkzeug
Unit Tests	Testen einzelner Python-Funktionen (z.B. Konfliktprüfung)	pytest
API-Tests	Testen aller REST-Endpunkte auf korrekte Antworten	Postman / pytest
Manuelle Tests	UI-Tests der React-Oberfläche im Browser	Manuell / Chrome DevTools
Integrationstests	Zusammenspiel Frontend ↔ API ↔ Datenbank	Manuell

7.2 Testfälle (Auswahl)

Testfall	Erwartetes Ergebnis	Status
Raum doppelt buchen	Fehlermeldung: Raum bereits belegt	✓ Bestanden
Trainer mit Terminüberschneidung	Konflikt wird erkannt und gemeldet	✓ Bestanden
Kursplan für 1 Schuljahr generieren	Plan wird vollständig und konfliktfrei erstellt	✓ Bestanden
Termin manuell verschieben	Neuer Termin wird gespeichert, Konfliktcheck läuft	✓ Bestanden
Ungültige API-Anfrage (fehlende Felder)	400 Bad Request mit Fehlerbeschreibung	✓ Bestanden

© 2025 – Auszubildende Programmierprojekt

8. Fazit & Reflexion

8.1 Ergebnisse

Das Raumbuchungssystem wurde innerhalb von zwei Wochen erfolgreich umgesetzt. Alle zentralen Anforderungen wurden implementiert:

- Automatische, konfliktfreie Kursplan-Generierung für ein Schuljahr
- Vollständige CRUD-Verwaltung von Trainern, Räumen, Modulen und Azubis
- Manuelle Anpassung und Erweiterung des generierten Plans
- Übersichtliche Kalenderansicht im Frontend

8.2 Herausforderungen

Im Projektverlauf traten folgende Herausforderungen auf, die das Team gemeinsam lösen musste:

- Komplexität des Planungsalgorithmus bei vielen parallelen Constraints
- Abstimmung der API-Schnittstellen zwischen Frontend- und Backend-Team
- Zeitmanagement innerhalb des engen zweiwöchigen Rahmens

8.3 Lessons Learned

Das Projekt hat dem Team wertvolle Erfahrungen in der agilen Zusammenarbeit, der API-Entwicklung mit Flask und der komponentenbasierten Entwicklung mit React vermittelt. Für zukünftige Projekte würde das Team früher mit gemeinsamen API-Verträgen (z.B. OpenAPI-Spec) starten, um Integrationsprobleme zu vermeiden.

Anhang

A. Verwendete Bibliotheken

Bibliothek	Version	Verwendung
Flask	3.x	REST-API Framework
SQLAlchemy	2.x	ORM für SQLite-Datenbankzugriff
Flask-CORS	4.x	Cross-Origin Requests erlauben
React	18.x	Frontend UI-Framework
TypeScript	5.x	Typisiertes JavaScript
Axios	1.x	HTTP-Client im Frontend
Faker (Python)	19.x	Generierung von Testdaten

© 2025 – Auszubildende Programmierprojekt

B. Projektstruktur

Das Repository ist wie folgt strukturiert:

- backend/ – Python Flask Anwendung
- backend/models/ – Datenbankmodelle (SQLAlchemy)
- backend/controllers/ – REST-Controller und Routen
- backend/services/ – Geschäftslogik inkl. Planungsalgorithmus
- backend/database/ – SQLite-Datenbankdatei und Seed-Skripte
- frontend/ – React + TypeScript Anwendung
- frontend/src/components/ – Wiederverwendbare UI-Komponenten
- frontend/src/pages/ – Seitenansichten (Kalender, Verwaltung)
- frontend/src/api/ – API-Aufrufe und Typdefinitionen